

Informe Técnico: FNBC y Desnormalización Controlada en Food Store

Cátedra: Base de Datos II · **Comisión:** 2PRO4 · **Docente:** Sergio Neira

Integrantes: Tiziano Valentini, Juan Martin Garcia

Entregables asociados (misma carpeta): `tp_fnbc_control_lote.sql`, `tp_preparar_datos_hoy.sql`, `tp_desnormalizacion_top_categorias.sql`, `salida_parte1.txt`, `salida_parte2.txt`, `volumen_base.txt`

Parte 1: Análisis y aplicación de la FNBC (`control_lote_almacen`)

1.1. Dependencias funcionales (consigna 4.2.a)

Sobre $R(\text{LoteID}, \text{DepositoID}, \text{ResponsableControlID})$, a partir de la regla de negocio de logística:

- «Para un lote y un depósito interviniente dados, el responsable de control queda unívocamente determinado»: **DF1: $\{\text{LoteID}, \text{DepositoID}\} \rightarrow \text{ResponsableControlID}$**
- «Cada responsable de control pertenece a un único depósito»: **DF2: $\text{ResponsableControlID} \rightarrow \text{DepositoID}$**

1.2. Clausuras, claves candidatas y atributos primos (4.2.b)

Clausuras de los subconjuntos razonables, usando $F = \{\text{DF1}, \text{DF2}\}$:

Conjunto X	X^+	¿Determina R?
$\{\text{LoteID}\}$	$\{\text{LoteID}\}$	No
$\{\text{DepositoID}\}$	$\{\text{DepositoID}\}$	No
$\{\text{ResponsableControlID}\}$	$\{\text{ResponsableControlID}, \text{DepositoID}\}$ (por DF2)	No
$\{\text{DepositoID}, \text{ResponsableControlID}\}$	$\{\text{DepositoID}, \text{ResponsableControlID}\}$	No
$\{\text{LoteID}, \text{DepositoID}\}$	$\{\text{LoteID}, \text{DepositoID}, \text{ResponsableControlID}\}$ (por DF1)	Sí
$\{\text{LoteID}, \text{ResponsableControlID}\}$	$\{\text{LoteID}, \text{ResponsableControlID}, \text{DepositoID}\}$ (por DF2)	Sí

Ninguno de los subconjuntos de un solo atributo determina R, por lo tanto las dos claves de dos atributos son **minimales**:

- Claves candidatas:** $CC = \{ \{\text{LoteID}, \text{DepositoID}\}, \{\text{LoteID}, \text{ResponsableControlID}\} \}$. La PK declarada en el DDL es la primera.

- **Atributos primos:** LotelID, DepositoID y ResponsableControlID (cada uno pertenece a alguna clave candidata).
- **Atributos no primos:** ninguno.

1.3. Demostración de violación de FNBC (4.2.c)

Definición. R está en FNBC si y solo si, para toda dependencia funcional no trivial $X \rightarrow Y$ válida en R, X es superclave de R.

- DF1: $\{\text{LotelID}, \text{DepositoID}\}^+ = R$, el determinante es superclave. Cumple.
- DF2: $\{\text{ResponsableControlID}\}^+ = \{\text{ResponsableControlID}, \text{DepositoID}\} \neq R$. El determinante no contiene a LotelID, por lo que **no es superclave**, y DF2 es no trivial (DepositoID no está en su lado izquierdo).

Veredicto: `control_lote_almacen` **no está en FNBC**; la dependencia violatoria es **ResponsableControlID $\rightarrow$ DepositoID**.

Observación: la relación sí está en 3FN, porque el consecuente DepositoID es un atributo primo (condición alternativa de 3FN que FNBC no admite). Por eso la violación pasa desapercibida si solo se verifica 3FN.

1.4. Anomalías sobre la instancia provista (4.2.d)

Instancia: (501, 30, 801), (502, 30, 801), (503, 31, 802). Cada escenario se reproduce con SQL en la sección 2 de `tp_fnbc_control_lote.sql` (dentro de transacciones que se revierten).

- **Insertión.** El inspector 803 se incorpora a la dotación del depósito 32. Ese hecho ($803 \rightarrow 32$) no puede guardarse sin asociarlo a un lote, porque `lote_id` forma parte de la PK y es **NOT NULL**: se debería inventar un lote o esperar a que exista una inspección.
- **Borrado.** Si se cancela el control del lote 503 y se elimina la fila (503, 31, 802), se pierde el único registro de que el responsable 802 pertenece al depósito 31, aunque ese dato maestro no tenía relación con el lote.
- **Actualización.** El par $801 \rightarrow$ depósito 30 está repetido en las filas de los lotes 501 y 502. Reasignar a 801 al depósito 32 exige actualizar ambas filas. Si solo se actualiza una (error o concurrencia), 801 queda en dos depósitos a la vez y **la PK (lote, depósito) no lo impide**, porque DF2 no está protegida por ninguna restricción.

1.5. Descomposición sin pérdida (4.2.e)

Algoritmo de descomposición sobre la DF violatoria $X \rightarrow Y$ con $X = \text{ResponsableControlID}$, $Y = \text{DepositoID}$:

- **R1(ResponsableControlID, DepositoID)** = $X \cup Y \rightarrow$ tabla `responsable_deposito`, PK = $\{\text{ResponsableControlID}\}$.
- **R2(LotelID, ResponsableControlID)** = $R - Y \rightarrow$ tabla `asignacion_control_lote`, PK = $\{\text{LotelID}, \text{ResponsableControlID}\}$.

```
CREATE TABLE responsable_deposito (
  responsable_control_id BIGINT PRIMARY KEY REFERENCES usuario(id),
  deposito_id            BIGINT NOT NULL REFERENCES deposito(id)
);
```

```
CREATE TABLE asignacion_control_lote (
  lote_id          BIGINT NOT NULL REFERENCES lote(id),
  responsable_control_id BIGINT NOT NULL REFERENCES
responsable_deposito(responsable_control_id),
  PRIMARY KEY (lote_id, responsable_control_id)
);

CREATE VIEW vista_control_lote_almacen_fnbc AS
SELECT lote_id, deposito_id, responsable_control_id
FROM asignacion_control_lote
NATURAL JOIN responsable_deposito;
```

Ambas relaciones están en FNBC: en R1 el único determinante no trivial (ResponsableControlID) es su clave; R2 solo tiene dependencias triviales.

1.6. Reunión sin pérdida y migración (4.2.f)

Criterio. Una descomposición de R en (R1, R2) es sin pérdida respecto de F si y solo si $R1 \cap R2 \rightarrow R1$ o $R1 \cap R2 \rightarrow R2$ se deduce de F, es decir, si el atributo común es superclave de R1 o de R2.

- $R1 \cap R2 = \{\text{ResponsableControlID}\}$.
- $\{\text{ResponsableControlID}\}^+ = \{\text{ResponsableControlID}, \text{DepositoID}\} = R1$, por lo que es superclave (de hecho, clave) de R1.

Por lo tanto la reunión natural $R1 \bowtie R2$ reconstruye exactamente R, sin tuplas espurias. La migración (`INSERT ... SELECT DISTINCT` desde `control_lote_almacen`) y la verificación con `EXCEPT` en ambos sentidos se ejecutan al final de `tp_fnbc_control_lote.sql`. Salida obtenida:

```
food_store=# SELECT * FROM control_lote_almacen EXCEPT SELECT * FROM
vista_control_lote_almacen_fnbc;
lote_id | deposito_id | responsable_control_id
-----+-----+-----
(0 rows)

food_store=# SELECT * FROM vista_control_lote_almacen_fnbc EXCEPT SELECT * FROM
control_lote_almacen;
lote_id | deposito_id | responsable_control_id
-----+-----+-----
(0 rows)
```

Verificación de cardinalidad y resolución de anomalías (salida de `tp_fnbc_control_lote.sql`):

```
filas_original | filas_vista
-----+-----
3 | 3
(1 fila)
```

NOTICES de demostración de anomalías y protección de DF1:

- **Anomalía de Inserción:** NOTICE: ANOMALÍA DE INSERCIÓN: no se puede registrar que 803 pertenece al depósito 32 sin un lote (el valor nulo en la columna «lote_id» de la relación «control_lote_almacen» viola la restricción “not-null”).
- **Anomalía de Borrado:** Resuelta (la vista sobre el esquema descompuesto preserva la tupla de responsable_deposito independiente del lote).
- **Anomalía de Actualización:** Resuelta (separación de la dependencia funcional en responsable_deposito).
- **Protección de DF1:** NOTICE: DF1 protegida: DF1 violada: el lote 501 ya tiene otro responsable de control en el mismo depósito

Costo de la descomposición: preservación de dependencias

La descomposición es sin pérdida, pero **no preserva DF1**: DF1 involucra atributos de R1 y R2 a la vez (LoteID por un lado, DepósitoID por el otro), así que ninguna restricción **PRIMARY KEY** o **FOREIGN KEY** puede garantizarla. Sin protección, el lote 501 podría tener asignados dos responsables del mismo depósito (801 y otro del depósito 30), que es un estado inválido según DF1. Es el costo clásico de llevar un esquema a FNBC (no siempre es posible conservar todas las dependencias). Se lo resolvió con dos disparadores en el script (`trg_validar_df1_asignacion`, sobre altas/cambios de asignación, y `trg_validar_df1_cambio_deposito`, sobre el traslado de un responsable a otro depósito); el script incluye una prueba (sección 8.d) donde un segundo responsable del mismo depósito para el mismo lote es rechazado. Las tres anomalías de 1.4 se demuestran también como **resueltas** en la sección 8 del script.

Parte 2: Desnormalización controlada (top 5 categorías del día)

2.1. Consulta de partida y adaptación al esquema real

La consulta de la consigna usa nombres genéricos. El repositorio (TP1) tiene otro esquema, así que se adaptó sin cambiar su semántica:

Consigna	Esquema real de Food Store
ped.fecha = CURRENT_DATE	ped.fecha_hora >= CURRENT_DATE AND ped.fecha_hora < CURRENT_DATE + 1 (fecha_hora es TIMESTAMPTZ; el rango equivale a fecha_hora::date = CURRENT_DATE y permite usar índices)
dp.subtotal	dp.cantidad * dp.precio_unitario (no existe columna subtotal)
producto_id, pedido_id, pr.id, c.id, c.nombre	id_producto, id_pedido, id_producto, id_categoria, nombre_categoria
eliminado	existe en Pedido y Detalle_Pedido por TP5/soft_delete.sql

```
SELECT c.nombre_categoria AS categoria,
       SUM(dp.cantidad * dp.precio_unitario) AS total_vendido
```

```

FROM Detalle_Pedido dp
JOIN Producto pr ON pr.id_producto = dp.id_producto
JOIN Categoria c ON c.id_categoria = pr.id_categoria
JOIN Pedido ped ON ped.id_pedido = dp.id_pedido
WHERE ped.fecha_hora >= CURRENT_DATE AND ped.fecha_hora < CURRENT_DATE + 1
AND dp.eliminado = FALSE AND ped.eliminado = FALSE
GROUP BY c.nombre_categoria
ORDER BY total_vendido DESC
LIMIT 5;

```

Preparación de datos y volumen real. La base contiene **50.005 productos**, **12 categorías**, **20.003 clientes**, **200.003 pedidos** y **400.011 detalles**. Con `tp_preparar_datos_hoy.sql` se reasignan a la fecha actual **10.239 pedidos** y **20.478 detalles** sobre la copia de trabajo.

2.2. Medición «antes» (5.2.a)

```

Limit (cost=10377.00..10377.01 rows=5 width=44) (actual time=83.281..88.845
rows=5.00 loops=1)
  Buffers: shared hit=4355
  -> Sort (cost=10377.00..10377.03 rows=12 width=44) (actual
time=83.280..88.844 rows=5.00 loops=1)
    Sort Key: (sum(((dp.cantidad)::numeric * dp.precio_unitario))) DESC
    Sort Method: top-N heapsort Memory: 25kB
    Buffers: shared hit=4355
    -> Finalize GroupAggregate (cost=10374.22..10376.80 rows=12 width=44)
(actual time=83.255..88.835 rows=12.00 loops=1)
      Group Key: c.nombre_categoria
      Buffers: shared hit=4355
      -> Gather Merge (cost=10374.22..10376.50 rows=20 width=44)
(actual time=83.248..88.819 rows=24.00 loops=1)
        Workers Planned: 1
        Workers Launched: 1
        Buffers: shared hit=4355
        -> Sort (cost=9374.21..9374.24 rows=12 width=44) (actual
time=68.474..68.478 rows=12.00 loops=2)
          Sort Key: c.nombre_categoria
          Sort Method: quicksort Memory: 26kB
          Buffers: shared hit=4355
          Worker 0: Sort Method: quicksort Memory: 26kB
          -> Partial HashAggregate (cost=9373.84..9373.99
rows=12 width=44) (actual time=68.431..68.437 rows=12.00 loops=2)
            Group Key: c.nombre_categoria
            Batches: 1 Memory Usage: 32kB
            Buffers: shared hit=4347
            Worker 0: Batches: 1 Memory Usage: 32kB
            -> Hash Join (cost=3674.45..9314.78 rows=5906
width=22) (actual time=15.881..64.502 rows=10239.00 loops=2)
              Hash Cond: (pr.id_categoria =
c.id_categoria)

              Buffers: shared hit=4347
              -> Hash Join (cost=3673.18..9292.53

```

```

rows=5906 width=18) (actual time=15.740..62.633 rows=10239.00 loops=2)
      Hash Cond: (dp.id_producto =
pr.id_producto)
      Buffers: shared hit=4345
      -> Parallel Hash Join
(cost=2032.07..7635.92 rows=5906 width=18) (actual time=2.095..45.359
rows=10239.00 loops=2)
      Hash Cond: (dp.id_pedido =
ped.id_pedido)
      Buffers: shared hit=3313
      -> Parallel Seq Scan on
detalle_pedido dp (cost=0.00..5295.01 rows=117651 width=26) (actual
time=0.012..26.296 rows=200005.50 loops=2)
      Filter: (NOT eliminado)
      Buffers: shared hit=2942
      -> Parallel Hash
(cost=1958.24..1958.24 rows=5906 width=8) (actual time=1.980..1.980 rows=5119.50
loops=2)
      Buckets: 16384 Batches:
1 Memory Usage: 544kB
      Buffers: shared hit=371
      -> Parallel Bitmap Heap
Scan on pedido ped (cost=271.35..1958.24 rows=5906 width=8) (actual
time=0.533..2.414 rows=10239.00 loops=1)
      Recheck Cond:
((fecha_hora >= CURRENT_DATE) AND (fecha_hora < (CURRENT_DATE + 1)))
      Filter: (NOT
eliminado)
      Heap Blocks:
exact=321
      Buffers: shared
hit=371
      -> Bitmap Index
Scan on idx_pedido_fecha_cliente (cost=0.00..268.84 rows=10041 width=0) (actual
time=0.477..0.478 rows=10250.00 loops=1)
      Index Cond:
((fecha_hora >= CURRENT_DATE) AND (fecha_hora < (CURRENT_DATE + 1)))
      Index
Searches: 1
      Buffers:
shared hit=50
      -> Hash (cost=1016.05..1016.05
rows=50005 width=16) (actual time=13.482..13.482 rows=50005.00 loops=2)
      Buckets: 65536 Batches: 1
Memory Usage: 2856kB
      Buffers: shared hit=1032
      -> Seq Scan on producto pr
(cost=0.00..1016.05 rows=50005 width=16) (actual time=0.048..6.666 rows=50005.00
loops=2)
      Buffers: shared hit=1032
      -> Hash (cost=1.12..1.12 rows=12
width=20) (actual time=0.130..0.131 rows=12.00 loops=2)
      Buckets: 1024 Batches: 1 Memory
Usage: 9kB

```

```

Buffers: shared hit=2
-> Seq Scan on categoria c
(cost=0.00..1.12 rows=12 width=20) (actual time=0.121..0.122 rows=12.00 loops=2)
    Buffers: shared hit=2

Planning Time: 0.567 ms
Execution Time: 89.186 ms

```

Análisis de la medición:

- **Execution Time:** 89.186 ms (en caché caliente).
- **Nodo dominante:** El mayor tiempo propio se concentra en el nodo **Parallel Seq Scan on detalle_pedido dp** (tiempo propio de ~26.296 ms por worker sobre 200.005.5 filas brutas evaluadas para descartar los inactivos) y en la unificación a través de los joins sucesivos con **Producto** y **Categoria**.

2.3. Patrón elegido y justificación (5.2.b)

Se eligió **columna precalculada con disparadores**: **Detalle_Pedido.categoria_nombre**. La decisión se apoya en la medición de 2.2 (**Execution Time de 89.186 ms**, con predominio del **Parallel Seq Scan** sobre **Detalle_Pedido** y los joins de catálogo): el costo se concentra en recorrer y unir tablas grandes para resolver, en cada ejecución, un dato —el nombre de categoría de cada línea— que cambia muy rara vez y que el panel consulta muchas veces por minuto; una vista materializada no sirve porque el panel necesita datos «en tiempo real» y la vista solo refleja el último **REFRESH**. La sincronización la garantizan tres disparadores que cubren todos los caminos por los que cambia la fuente de verdad (alta o cambio de producto de un detalle, renombre de una categoría y cambio de categoría de un producto); en el alta el valor siempre se recalcula en **BEFORE INSERT**, así que el cliente no puede forzar uno falso, y la columna es **NOT NULL**. Además, un script de auditoría compara la columna contra **Producto ⋈ Categoria** y debe devolver cero filas. La estructura es **reversible sin pérdida**: **categoria_nombre** es un dato derivado, por lo que **DROP COLUMN** y la eliminación de los disparadores restituyen el esquema original sin perder información (bloque «REVERSIÓN» del script). El costo asumido es espacio extra (~400.011 valores repetidos), escrituras algo más caras y una actualización masiva si se renombra una categoría con muchas ventas.

2.4. Implementación (5.2.c)

Script completo: **tp_desnormalizacion_top_categorias.sql**. Resumen:

1. **ALTER TABLE Detalle_Pedido ADD COLUMN categoria_nombre VARCHAR(80)**, migración con **UPDATE ... FROM Producto JOIN Categoria** y luego **SET NOT NULL**.
2. **trg_dp_sync_categoria_nombre** (**BEFORE INSERT OR UPDATE OF id_producto, categoria_nombre** en **Detalle_Pedido**): recalcula siempre la columna desde la fuente.
3. **trg_categoria_propagar_nombre** (**AFTER UPDATE OF nombre_categoria** en **Categoria**): propaga el renombre.
4. **trg_producto_propagar_categoria** (**AFTER UPDATE OF id_categoria** en **Producto**): propaga el cambio de categoría de un producto.

Los triggers se crean después de la migración masiva y conviven con los de TP2 (**trg_validar_stock_pedido**, **trg_bloquear_modificacion_detalle_pedido**) y TP5 (**trg_devolver_stock_al_anular**), que no tocan esta columna.

2.5. Consulta sobre la estructura desnormalizada y comparación (5.2.d)

```
SELECT dp.categoria_nombre AS categoria,
       SUM(dp.cantidad * dp.precio_unitario) AS total_vendido
FROM Detalle_Pedido dp
JOIN Pedido ped ON ped.id_pedido = dp.id_pedido
WHERE ped.fecha_hora >= CURRENT_DATE AND ped.fecha_hora < CURRENT_DATE + 1
      AND dp.eliminado = FALSE AND ped.eliminado = FALSE
GROUP BY dp.categoria_nombre
ORDER BY total_vendido DESC
LIMIT 5;
```

Las tablas **Producto** y **Categoria** desaparecen del plan (de 4 tablas a 2; **Pedido** se conserva porque aporta la fecha y el estado de anulación del pedido, que no se replicaron a propósito).

```
Limit (cost=11737.32..11737.33 rows=5 width=44) (actual time=50.923..56.632
rows=5.00 loops=1)
  Buffers: shared hit=6891
  -> Sort (cost=11737.32..11737.35 rows=12 width=44) (actual
time=50.922..56.630 rows=5.00 loops=1)
    Sort Key: (sum(((dp.cantidad)::numeric * dp.precio_unitario))) DESC
    Sort Method: top-N heapsort Memory: 25kB
    Buffers: shared hit=6891
    -> Finalize GroupAggregate (cost=11733.37..11737.12 rows=12 width=44)
(actual time=50.891..56.621 rows=12.00 loops=1)
      Group Key: dp.categoria_nombre
      Buffers: shared hit=6891
      -> Gather Merge (cost=11733.37..11736.75 rows=29 width=44)
(actual time=50.883..56.600 rows=36.00 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=6891
        -> Sort (cost=10733.35..10733.38 rows=12 width=44) (actual
time=33.295..33.298 rows=12.00 loops=3)
          Sort Key: dp.categoria_nombre
          Sort Method: quicksort Memory: 26kB
          Buffers: shared hit=6891
          Worker 0: Sort Method: quicksort Memory: 26kB
          Worker 1: Sort Method: quicksort Memory: 26kB
          -> Partial HashAggregate (cost=10732.98..10733.13
rows=12 width=44) (actual time=33.254..33.259 rows=12.00 loops=3)
            Group Key: dp.categoria_nombre
            Batches: 1 Memory Usage: 32kB
            Buffers: shared hit=6875
            Worker 0: Batches: 1 Memory Usage: 32kB
            Worker 1: Batches: 1 Memory Usage: 32kB
            -> Parallel Hash Join (cost=2032.07..10649.30
rows=8368 width=22) (actual time=2.039..30.881 rows=6826.00 loops=3)
              Hash Cond: (dp.id_pedido = ped.id_pedido)
              Buffers: shared hit=6875
```

```

                                -> Parallel Seq Scan on detalle_pedido dp
(cost=0.00..8179.71 rows=166671 width=30) (actual time=0.871..19.427
rows=133337.00 loops=3)
                                Filter: (NOT eliminado)
                                Buffers: shared hit=6513
                                -> Parallel Hash  (cost=1958.24..1958.24
rows=5906 width=8) (actual time=1.100..1.101 rows=3413.00 loops=3)
                                Buckets: 16384  Batches: 1  Memory
Usage: 544kB
                                Buffers: shared hit=362
                                -> Parallel Bitmap Heap Scan on
pedido ped  (cost=271.35..1958.24 rows=5906 width=8) (actual time=0.475..2.022
rows=10239.00 loops=1)
                                Recheck Cond: ((fecha_hora >=
CURRENT_DATE) AND (fecha_hora < (CURRENT_DATE + 1)))
                                Filter: (NOT eliminado)
                                Heap Blocks: exact=312
                                Buffers: shared hit=362
                                -> Bitmap Index Scan on
idx_pedido_fecha_cliente  (cost=0.00..268.84 rows=10041 width=0) (actual
time=0.428..0.428 rows=10239.00 loops=1)
                                Index Cond: ((fecha_hora
>= CURRENT_DATE) AND (fecha_hora < (CURRENT_DATE + 1)))
                                Index Searches: 1
                                Buffers: shared hit=50

Planning Time: 0.331 ms
Execution Time: 56.685 ms

```

	Antes (4 tablas)	Después (columna desnormalizada)
Execution Time	89.186 ms	56.685 ms (-36.44%)
Nodo dominante	Parallel Seq Scan on Detalle_Pedido + Joins de catálogo	Parallel Seq Scan on Detalle_Pedido (dp + ped)
Tablas en el plan	Detalle_Pedido, Producto, Categoria, Pedido	Detalle_Pedido, Pedido

Verificación de equivalencia: los resultados de «RESULTADO NORMALIZADO» y «RESULTADO DESNORMALIZADO» en **salida_parte2.txt** son idénticos (**verificado fila por fila**).

2.6. Auditoría de sincronización (5.2.e)

```

SELECT dp.id_pedido, dp.id_producto,
       dp.categoria_nombre AS valor_desnormalizado,
       c.nombre_categoria  AS valor_fuente_verdad
FROM Detalle_Pedido dp
JOIN Producto  pr ON pr.id_producto = dp.id_producto

```

```
JOIN Categoria c ON c.id_categoria = pr.id_categoria
WHERE dp.categoria_nombre IS DISTINCT FROM c.nombre_categoria;
```

```
=== AUDITORÍA (esperado: 0 filas) ===
 id_pedido | id_producto | valor_desnormalizado | valor_fuente_verdad
-----+-----+-----+-----
(0 rows)
```

El script incluye además tres pruebas de sincronización con **ROLLBACK** (P1: alta con valor forzado falso; P2: renombre de categoría; P3: cambio de categoría de un producto). La auditoría da 0 filas después de P2 y P3.

Salidas de las pruebas P1, P2 y P3:

- **P1 (INSERT con valor sobrescrito por el trigger — Producto 1 es 'Pizzas'):**

```
=== P1: INSERT con categoria_nombre forzada a un valor falso ===
BEGIN
 id_pedido | id_producto | categoria_nombre
-----+-----+-----
      200004 |           1 | Pizzas
(1 row)

INSERT 0 1
ROLLBACK
```

- **P2 (Renombre de categoría propagado):**

```
=== P2: renombre de categoría 1 ===
BEGIN
UPDATE 1
 detalles_con_nombre_nuevo
-----
                        33387
(1 row)

 filas_desincronizadas
-----
                        0
(1 row)

ROLLBACK
```

- **P3 (Cambio de categoría de un producto propagado — Producto 1 pasa a Categoría 2 = 'Bebidas'):**

```

=== P3: producto 1 pasa a la categoría 2 ===
BEGIN
UPDATE 1
  categoria_nombre
-----
  Bebidas
(1 row)

  filas_desincronizadas
-----
                                0
(1 row)

ROLLBACK

```

2.7. Alcance y límites

- La comparación mide contra los índices que ya tiene el proyecto (TP1, TP3 y TP5). El **Parallel Seq Scan** sobre **Detalle_Pedido** sigue presente porque la consulta agrupa y suma todos los detalles activos del día sin un índice filtrado por fecha en detalle (la fecha reside en **Pedido**).
- Los disparadores de propagación (renombrar de categoría o cambio de categoría de un producto) actualizan potencialmente decenas de miles de filas en una sola transacción: son operaciones raras de catálogo, pero conviene ejecutarlas fuera de horario pico.
- Semántica: el reporte refleja la categoría **vigente** del producto (igual que la consulta normalizada). Si se quisiera la categoría al momento de la venta habría que dejar de propagar los cambios y se perdería la equivalencia con la consulta original.